

SVD of Zero-Padded Matrices via Mask Augmentation

August 2026

1 Problem setup and notation

Let $A \in \mathbb{R}^{n \times m}$ (with m small, n possibly large) be embedded into a fixed-shape padded matrix

$$A_{\text{pad}} \in \mathbb{R}^{N \times M}, \quad N = n + P_r, \quad M = m + P_c,$$

where P_r row positions and P_c column positions — at *arbitrary* (possibly interior) locations — are identically zero padding. Denote by $\mathcal{S}_r \subset \{1, \dots, N\}$, $|\mathcal{S}_r| = P_r$, the set of pad-row indices and $\mathcal{S}_c \subset \{1, \dots, M\}$, $|\mathcal{S}_c| = P_c$, the set of pad-column indices. In a batched (JAX) setting, P_r and P_c are the *maximum* pad counts across the batch, so all shapes below are static; the masks $\mathcal{S}_r, \mathcal{S}_c$ are runtime data (boolean vectors of static length) and cause no recompilation.

The failure mode. If A has numerical rank $k < m$, the left null space of A_{pad} contains both genuine null directions of A and the pad coordinates $\text{span}\{e_i : i \in \mathcal{S}_r\}$. A black-box SVD returns an arbitrary orthonormal basis of this degenerate subspace, so the null-space columns of U generically place mass on pad rows; restricted to the data rows, U is then non-orthonormal and possibly rank deficient.

The output contract. The correct requirement is

$$\text{pad} \rightarrow \text{svd} \rightarrow \text{unpad} \equiv \text{svd of the unpadded } A,$$

where **unpad** slices with the masks in their *original* (possibly interior) locations. Concretely: the *first m triplets* of the returned factorization must be a valid economy SVD of A — k positive triplets plus $m - k$ data-null ones — with $U[:, : m]$ zero on pad rows and $V[:, : m]$ zero on pad-column coordinates; the remaining $M - m$ *don't-care* triplets carry $\sigma = 0$ and may fill the pad region (or data rows) freely, provided the full output is still a genuine SVD of A_{pad} . Two consequences. First, feasibility requires only $n \geq m$ — exactly the condition the unpadded economy SVD needs — so $n < M$ (more total columns than data rows) is a supported regime, not a failure mode. Demanding instead that *all* M columns of U avoid pads would require $n \geq M$; that stronger contract is what early drafts of Method D assumed, and it is both unnecessary and unachievable in general. Second, since any valid economy SVD of A has arbitrary orthonormal completions in its null slots, the contract constrains null columns only in support and orthonormality, never in direction.

Five methods. All fixes below hand the pad masks to the computation *as data*, with all shapes static. **Method A (augmentation, §2–4)** breaks the degeneracy inside the SVD by pinning pad coordinates to a separated singular value; it is clean and tolerance-free but its cost grows with the pad count, so it suits $P_r, P_c \ll n$. **Method B (post-hoc masked completion, §5)** leaves the SVD untouched and rebuilds only the degenerate columns afterward; its cost is independent of the pad count, making it the right choice when $P_r \sim N$ is possible, at the price of an explicit rank tolerance. **Method C (masked noise, §6)** and its refinement **C'** (§7) trade exactness for simplicity: a controlled, precision-dependent error of order $\sqrt{\varepsilon}$ (respectively $\varepsilon^{2/3}$) in exchange for minimal code changes, with the side benefit of regularizing the SVD derivative in differentiable pipelines. **Method D (sketch-project, §8)** combines the strengths of A and B — exact, tolerance-free, pad-count-independent — by exploiting the fact that matrix–vector products with A_{pad} are structurally blind to pad rows; it is the recommended default.

2 Method A: the augmented matrix

Choose a separation constant

$$c = 4 \|A_{\text{pad}}\|_F \quad (\text{any bound with margin on both sides of the filter works}).$$

Warning on the margin: the natural-looking choice $c = 2 \|A_{\text{pad}}\|_2$ is *fragile*: the filter threshold $c/2$ then sits exactly at σ_{max} , and a computed σ_{max} one ulp high is misclassified as a pinned triplet — silently deleting the largest singular value. A rank-1 matrix with a cleanly representable norm triggers this deterministically. With $c = 4 \|A_{\text{pad}}\|_F$ the threshold $c/2 = 2 \|A_{\text{pad}}\|_F$ has $\|A\|$ -sized margins to both the data spectrum and the pins. Define the one-hot mask matrices

$$\begin{aligned} E_r &\in \mathbb{R}^{N \times P_r}, & (E_r)_{i\ell} = 1 &\iff i = \ell\text{-th pad row}, \\ E_c &\in \mathbb{R}^{M \times P_c}, & (E_c)_{j\ell} = 1 &\iff j = \ell\text{-th pad column}. \end{aligned}$$

(If an instance has fewer pads than the static maximum, surplus columns of E_r, E_c are zero; they contribute harmless exact-zero singular triplets.) The **two-sided augmented matrix** is

$$A_{\text{aug}} = \begin{bmatrix} A_{\text{pad}} & c E_r \\ c E_c^\top & 0 \end{bmatrix} \in \mathbb{R}^{(N+P_c) \times (M+P_r)}.$$

For the row-pads-only case simply drop the second block row: $A_{\text{aug}} = [A_{\text{pad}} \ c E_r] \in \mathbb{R}^{N \times (M+P_r)}$.

Shape summary

Assume m small, n large, P_r, P_c small. All matrices are tall–skinny or small–square; *no $n \times n$ object is ever formed*.

Matrix	Shape	Size regime	Role
A_{pad}	$N \times M$	$n \times m$	padded input
E_r	$N \times P_r$	$n \times \text{small}$	row-pad mask (one-hot)
E_c	$M \times P_c$	$m \times \text{small}$	column-pad mask (one-hot)
A_{aug}	$(N+P_c) \times (M+P_r)$	$\sim n \times (m+\text{small})$	augmented input
Q	$(N+P_c) \times (M+P_r)$	$\sim n \times (m+\text{small})$	economy QR factor
R	$(M+P_r) \times (M+P_r)$	small square	SVD is computed here
U_R, Σ, V	$(M+P_r) \times (M+P_r)$	small square	SVD of R
$U = Q U_R$	$(N+P_c) \times (M+P_r)$	$\sim n \times (m+\text{small})$	left vectors, pre-filtering
U, V (final)	$(N+P_c) \times M, (M+P_r) \times M$	tall/small	after dropping c -triplets

3 Why it works

Because pad rows of A_{pad} are zero, $\text{col}(A_{\text{pad}}) \perp e_i$ for every $i \in \mathcal{S}_r$, and symmetrically for pad columns. Direct verification then shows that each pad row i (paired with its one-hot column in E_r) forms an *exact* singular triplet of A_{aug} with singular value c , and likewise each pad column. Thus:

1. The pad coordinates constitute the invariant subspace attached to the singular value c , which is separated from the data spectrum ($\sigma_{\max}(A_{\text{pad}}) \leq c/2$).
2. Every remaining singular vector — *including the $\sigma \approx 0$ null-space columns that caused the original problem* — is orthogonal to the c -cluster, hence supported on data rows/columns, up to a rounding error of order $\varepsilon c/(c - \sigma_{\max}) \sim \varepsilon$.
3. The degeneracy is broken *by construction*, for arbitrary pad positions, with no permutation bookkeeping and no branching on n .

4 Method A: complete algorithm

Step 1. Masks. From the runtime pad masks build E_r ($N \times P_r$) and E_c ($M \times P_c$), e.g. by `argsort` of the boolean mask followed by one-hot encoding; zero out surplus columns when an instance has fewer pads than the static maximum.

Step 2. Scale. Set $c = 4 \|A_{\text{pad}}\|_F$ (see the margin warning in §2; $2 \|A_{\text{pad}}\|$ puts the filter exactly at $\sigma_{\max}$). If $A_{\text{pad}} = 0$ is possible in the batch, use $c = 4 \|A_{\text{pad}}\|_F + \delta$ with a fixed $\delta > 0$ to keep $c > 0$.

Step 3. Augment. Form A_{aug} as above, shape $(N + P_c) \times (M + P_r)$.

Step 4. QR (Chan step). Compute the economy QR factorization $A_{\text{aug}} = QR$, with Q of shape $(N + P_c) \times (M + P_r)$ and R square of side $M + P_r$. Cost $O(N(M+P_r)^2)$.

Step 5. Small SVD. Compute $R = U_R \Sigma V^\top$ (`full_matrices=false`); this is a dense SVD of a small, statically-shaped square matrix. Cost $O((M+P_r)^3)$.

Step 6. Recombine. $U = Q U_R$. Cost $O(N(M+P_r)^2)$.

Step 7. Filter. Identify the pad triplets by $\sigma_i > c/2$; the data triplets satisfy $\sigma_i \leq c/2$. Discard the pad triplets and, if desired, `argsort` the survivors by σ and gather with `take_along_axis`. Exactly M triplets survive per instance (statically), yielding $U \in \mathbb{R}^{(N+P_c) \times M}$, $\Sigma \in \mathbb{R}^M$, $V \in \mathbb{R}^{(M+P_r) \times M}$.

Step 8. Restrict (optional). Rows of U indexed by $\mathcal{S}_r$ and rows of V indexed by $\mathcal{S}_c$ are zero to $O(\varepsilon)$; restricting U to data rows gives an orthonormal (to $O(\varepsilon)$) matrix of left singular vectors of A , including well-behaved null-space columns. Note that among the M surviving triplets, P_c correspond to the zero columns of A_{pad} (they have $\sigma = 0$ and right vectors supported on pad columns); drop these too if only the m -column SVD of A is wanted.

Costs. Total flops $O(N(M+P_r)^2 + (M+P_r)^3)$, linear in n ; peak memory is one tall-skinny matrix of width $M + P_r$ plus small squares. All shapes are static across the batch, so JAX compiles once and `vmaps` cleanly.

Caveat: cost scales with the pad count. Static shapes force P_r, P_c to be the *worst-case* pad counts over the batch, and the augmentation width cannot be compressed: the degenerate subspace being pinned has dimension exactly P_r , so any spectral-separation construction needs P_r extra columns. If batch sizes vary by a factor of two, $P_r \sim N/2$ is routine, and the QR of an $N \times (M+N/2)$ matrix costs $O(N^3)$ versus $O(NM^2)$ for the unaugmented problem. The asymmetry is that row-padding A_{pad} itself is cheap (linear in the tall dimension) while augmentation converts a *row*-pad count into extra *columns*, which is quadratic. In that regime use Method B.

5 Method B: post-hoc masked completion

Key observation

For every singular triplet with $\sigma > 0$, the left vector is determined by $u = A_{\text{pad}}v/\sigma$, and since the pad rows of A_{pad} are bitwise zero, those entries of u are *exactly* zero — at any σ , however tiny. Contamination therefore lives only in the $\sigma \approx 0$ columns, and those are an arbitrary orthonormal completion that need not come from the SVD at all: we may discard them and construct our own completion that respects the mask, after the fact, at width M .

Algorithm

Fix a rank tolerance $\tau \sim \varepsilon N \|A_{\text{pad}}\|$ (the standard SVD truncation threshold). All shapes below are static; the per-instance number of valid columns k lives entirely inside `argsort` permutations.

Step 1. SVD as usual. Economy SVD of A_{pad} (optionally Chan: QR then SVD of the $M \times M$ factor R). Keep Σ and V ; the shapes are exactly those of the unaugmented pipeline. Cost $O(NM^2 + M^3)$.

Step 2. Recompute valid columns. $U_{\text{valid}} = A_{\text{pad}}V \cdot \text{diag}(\sigma_i^{-1}[\sigma_i > \tau])$, i.e. safe division with columns $\sigma_i \leq \tau$ zeroed. (For $\sigma \in (0, \tau]$ the direction of $A_{\text{pad}}v/\sigma$ is noise-dominated even though its support is clean, so those columns are replaced as well.) The surviving k columns have exactly zero pad rows and are orthonormal to working accuracy. Cost $O(NM^2)$.

Step 3. Masked random block. Draw $G \in \mathbb{R}^{N \times M}$ from a fixed PRNG key (deterministic, traceable) and zero its pad rows; optionally also zero numerically-tiny data rows to enforce the stronger “avoid all zero rows” policy.

Step 4. Assemble and permute. Form $X = [U_{\text{valid}} \ G] \in \mathbb{R}^{N \times 2M}$, with the invalid (zeroed) columns of U_{valid} moved behind G via `argsort` on the validity flag, so the leading columns are the k good ones, then the M random ones, then zeros.

Step 5. Orthonormalize. Economy QR of X ; take the first M columns of Q as the final U . Cost $O(NM^2)$.

Step 6. Columns / V . Column pads get the mirror treatment on V (with $A_{\text{pad}}^\top$ and the column mask); the two sides are independent, so no pairing of P_r and P_c is needed.

Why it works

Every leading column of X has exactly zero pad rows, so the Householder reflectors of the QR have zero pad entries, and the first $k + M$ columns of Q are exactly zero on pads — including the $M - k$ completion columns. Those completion columns land in the left null space of A_{pad} automatically: they are orthogonal to $\text{col}(U_{\text{valid}}) = \text{col}(A_{\text{pad}})$ and supported on data rows, and any data-supported vector orthogonal to the column space is left-null.

Cost and caveats

Total cost is one extra $N \times 2M$ QR and a matmul beyond the baseline SVD, $O(NM^2)$ overall — the same order as the unaugmented pipeline and, crucially, *independent of P_r* . Two caveats:

1. **Feasibility.** Under the output contract, only the m genuine triplets must be data-row-supported, which requires $n \geq m$ — the unpadded problem’s own condition. The masked completion is applied to those slots; the $M - m$ don’t-care slots may be completed without the mask. (Demanding all M columns pad-free would require $n \geq M$.)
2. **Rank tolerance.** Unlike Method A, the valid/invalid split reintroduces a threshold τ . It is the standard truncation threshold applied downstream anyway, and misclassification is graceful: a borderline column replaced by completion still yields a legitimate orthonormal basis of the same near-null subspace.

6 Method C: masked noise injection

Methods A and B are (essentially) exact. Method C accepts a controlled, tunable error in exchange for maximal simplicity: one masked addition, an unmodified SVD call, no post-processing.

Construction

Let $G \in \mathbb{R}^{N \times M}$ be i.i.d. standard Gaussian from a fixed PRNG key, *with its pad rows and pad columns zeroed* — unmasked noise would lift the pad coordinates too and make the contamination deterministic, so this method needs the mask as data just like Methods A and B. Compute the SVD of

$$A' = A_{\text{pad}} + \eta G_{\text{masked}}, \quad t := \eta\sqrt{n} \text{ (the noise floor).}$$

For $n \geq M$ the data block is now generically full column rank: the true triplets are perturbed by at most t (Weyl), and the $M - k$ formerly-zero values lift into a cluster at level $\sim t$. (For $m \leq n < M$ the lift reaches only rank n ; the residual $M - n$ exact zeros are junk under the output contract, and sorting by σ still places the m genuine triplets — data-nulls lifted to $\sim t > 0$ — ahead of them.) Every left singular vector with $\sigma > 0$ is determined and equals $A'v/\sigma$, which has exactly zero pad rows in exact arithmetic.

Error channels

1. **Singular values:** $|\sigma_i(A') - \sigma_i(A)| \leq t$, uniformly. Values $\sigma_i \lesssim t$ are erased: t acts as a hard effective-rank threshold.
2. **Data singular vectors:** rotation $\sim t/\text{gap}_i$; clustered triplets are accurate only at the subspace level (as always).
3. **Pad contamination:** the computed factors are exact for $A' + \Delta$ with $\|\Delta\| \sim \varepsilon \|A\|$, and Δ ignores the mask; the computed left subspace tilts into the null space (which contains the pads) by $\sin \theta \sim \varepsilon \|A\| / t$.

Channels 1 and 3 trade off with fixed product $\varepsilon \|A\|^2$. Balancing gives $t^* = \sqrt{\varepsilon} \|A\|$ and *both* errors $\sim \sqrt{\varepsilon} \|A\|$ — the same half-precision loss as the Gram-matrix route, and for the same reason: the information distinguishing “zero” from “pad” has been compressed to a scale that rounding can reach. If tolerances are asymmetric (pad mass below δ , best possible σ), set $t = \varepsilon \|A\| / \delta$.

Precision	ε	Balanced error, plain (C)	With refinement (§7)
float64	$\sim 1.1 \cdot 10^{-16}$	$\sim 10^{-8} \ A\ $	$\sim 2 \cdot 10^{-11} \ A\ $
float32	$\sim 6 \cdot 10^{-8}$	$\sim 2.4 \cdot 10^{-4} \ A\ $	$\sim 1.5 \cdot 10^{-5} \ A\ $
bfloat16	$\sim 3.9 \cdot 10^{-3}$	$\sim 6 \cdot 10^{-2} \ A\ $	$\sim 2.5 \cdot 10^{-2} \ A\ $

Caveats

1. **Near-square data blocks.** The bounds assume the data block is genuinely tall. If $n - k \approx M - k$, the smallest lifted value behaves like $\eta/\sqrt{n}$ rather than $\eta\sqrt{n}$ (Rudelson–Vershynin small-ball: $\Pr[\sigma_{\min} \leq s\eta/\sqrt{n}] \lesssim s$), degrading channel 3 by a factor $\sim n$ with small but nonnegligible probability. Batches containing square-ish stragglers are exactly where this bites; inflate η or handle them separately.
2. **Lifted vectors are only a subspace.** Their internal gaps are tiny ($\sim t/(M - k)$ at best), so individual lifted columns churn from run to run; downstream code must consume them only as a basis of the near-null subspace.
3. **Differentiability bonus.** Inside a differentiable pipeline the lifting also regularizes the SVD derivative (the subgradient blowup at repeated σ is tamed by the same mechanism) — arguably the strongest genuine argument for Method C, since A and B do not help there.

7 Method C': Rayleigh–Ritz refinement of the noisy SVD

Construction

After the noisy SVD $A' = USV^\top$, stop trusting U, V as singular vectors and treat them only as orthonormal bases whose spans approximately capture the row/column spaces of A_{pad} . Project the *clean* matrix into those bases,

$$B = U^\top A_{\text{pad}} V \in \mathbb{R}^{M \times M}, \quad B = \bar{U} \bar{\Sigma} \bar{V}^\top,$$

and output $U_{\text{final}} = U\bar{U}$, $\Sigma = \bar{\Sigma}$, $V_{\text{final}} = V\bar{V}$, so that $A_{\text{pad}} \approx U_{\text{final}} \bar{\Sigma} V_{\text{final}}^\top$. Extra cost: two tall-skinny matmuls and one $M \times M$ SVD, $O(NM^2 + M^3)$ — negligible over the baseline.

Error analysis

Writing $A_{\text{pad}} = U^* \Sigma^* V^{*\top}$ exactly, one has $B = (U^\top U^*) \Sigma^* (V^{*\top} V)$ with outer factors having singular values in $[\cos \theta_{\max}, 1]$, giving the uniform *relative* bound

$$|\sigma_i(B) - \sigma_i| \lesssim \sigma_i (\theta_u^2 + \theta_v^2) + \varepsilon \|A\|, \quad \sin \theta \leq t/\sigma,$$

the $\varepsilon \|A\|$ floor coming from forming B in floating point. Projection errors are *quadratic* in the subspace angles: the singular-value error drops from $\sim t$ (Weyl, additive) to $\sim t^2/\sigma_i + \varepsilon \|A\|$, and data singular vectors improve similarly (one projection step roughly squares the angle, up to gap factors).

What does not improve: $U_{\text{final}} = U\bar{U}$ applies an $M \times M$ rotation to the columns of U , which can redistribute pad-row mass among columns but not remove it — channel 3 is invariant at $\sim \varepsilon \|A\|/t$. Likewise, values $\sigma_i \lesssim t$ remain unresolvable: their directions were scrambled into the lifted cluster before projection.

The improved tradeoff

With $\sigma_i \sim \|A\|$, balance $t^2/\|A\|$ against $\varepsilon \|A\|/t$: the optimum is $t^* = \varepsilon^{1/3} \|A\|$ with *both* errors $\sim \varepsilon^{2/3} \|A\|$ (table above). The uncertainty product improves from ε to $\varepsilon^{4/3}$. Since the σ error is quadratic in t , coarse noise is affordable when only singular values matter, paying only in pad cleanliness.

Two remarks. (i) The step can be iterated — it is one step of subspace iteration with the clean matrix, so angles keep squaring until the ε/gap floor — but the pad-mass floor $\varepsilon \|A\|/t$ and the $\sigma \lesssim t$ resolution limit are set at the noisy step and do not improve. (ii) If one additionally recomputes $U_{\text{final}} = A_{\text{pad}} V_{\text{final}} \bar{\Sigma}^{-1}$ (safe division) for $\bar{\sigma} > \tau$ and completes the null columns by masked orthonormalization, one reconstructs Method B with its random completion generated through an SVD instead of a QR: noise + refinement sits on a continuum between Method C and Method B.

8 Method D: sketch–project

The matvec principle

Any algorithm that touches A_{pad} only through products $A_{\text{pad}}x$ and $A_{\text{pad}}^\top y$ never sees the pad rows at all: $A_{\text{pad}}x$ has *bitwise* zero pad entries for every x , and $A_{\text{pad}}^\top y$ ignores the pad rows of y . Matrix-free methods (Lanczos/Golub–Kahan, LSQR, subspace iteration, randomized SVD) therefore generate left vectors in $\text{col}(A_{\text{pad}}) \perp \{e_i : i \in \mathcal{S}_r\}$ by construction. Methods B and C' are special cases of this observation; Method D is its direct use.

Construction (final, static-shape form)

Draw a fixed Haar-orthonormal sketch $\Omega \in \mathbb{R}^{M \times M}$ (QR of a fixed-seed Gaussian; $\kappa(\Omega) = 1$). Because Ω is square and invertible (probability 1), the sketch captures the column space *exactly*, $\text{col}(A_{\text{pad}}\Omega) = \text{col}(A_{\text{pad}})$ — randomized SVD with the approximation removed, which is why no oversampling parameter appears. Since $\text{col}(A_{\text{pad}}) \subseteq \text{span}(Q)$ implies $A_{\text{pad}} = Q Q^\top A_{\text{pad}}$, the SVD of the $M \times M$ projection $B = Q^\top A_{\text{pad}}$ already carries Σ and V ; no second sketch is needed. All mask-dependent operations below (**argsort**, **gathers**, **diagonal indicators**) are *traced runtime data with static shapes*: in JAX they compile once and never retrigger on a new mask pattern.

Step 1. Left basis. $Y = A_{\text{pad}}\Omega$ ($N \times M$; pad rows bitwise zero for any Ω). Permute pad rows to the bottom (**argsort** of the mask), economy QR, un-permute $\rightarrow Q$. If $n \geq M$, all pivots are data rows and every column of Q is bitwise zero on pads. If $n < M$, the degradation is *graceful*: the first n pivots are data rows; from step $n + 1$ the entire trailing submatrix lives on pad rows, hence is bitwise zero, hence $H = I$ throughout — so Q splits bitwise into n data-supported columns (a full orthonormal basis of the data-row subspace) followed by $M - n$ *exact* pad coordinate vectors. The surplus is flagged by the bitwise indicator $t_i = \|Q[\mathcal{S}_r, i]\| \in \{0, 1\}$.

Step 2. One augmented SVD, statically shaped. Form the $M \times 2M$ matrix $[B, c \text{diag}(t)]$ with $c = 4 \|A_{\text{pad}}\|_F$ (margin warning of §2 applies) and take its economy SVD, $W_1 \Sigma_a \cdot$. The static $\text{diag}(t)$ block replaces the dynamic-width augmentation: where $t_i = 1$ it pins the surplus coordinate at $\sigma = c$ exactly (row i of B is bitwise zero); where $t_i = 0$ it contributes a zero column, which is harmless because zero columns pollute only the *right* factor of this SVD — which is never used. The left null space of $[B, c \text{diag}(t)]$ is exactly the data-left-null (the diagonal block annihilates surplus components), so *every* kept left vector is clean, including the $\sigma = 0$ ones.

Step 3. Sort and assemble U, S . Order triplets by (pinned last, σ descending) via one **argsort** of the key where $(\sigma > c/2, -1, \sigma)$; zero the (exact-arithmetic-zero, $O(\varepsilon)$ in practice) surplus coordinates of the kept columns of W_1 ; set $U = QW_1$ and $S = \text{where}(i < m, \sigma_i, 0)$, with $m = M - p_c$ known from the column mask. The first m columns of U are bitwise zero on pad rows; the pinned columns come out as pure pad coordinate vectors (ideal don't-care fill).

Step 4. Rebuild V from U . Form $W = A_{\text{pad}}^\top U$. Every column is bitwise supported on data columns (pad columns of A_{pad} annihilate those rows), and $W = V\Sigma$ *exactly*, with Σ decreasing. The permuted QR of W (pad columns to trailing pivots) therefore *is* the factorization $V\Sigma$ up to column signs, recovered from $\text{sign}(\text{diag } R)$: the leading columns are the true right vectors — exactly, even through repeated- σ clusters, since orthonormal-times-diagonal is already in QR form — the middle columns are data-supported null completions (reflections of ε -level data-column content), and the trailing columns are exact pad coordinate vectors ($H = I$ on the bitwise-zero trailing block). This is the whole final V , in order, with no tolerance: it is Method B's completion realized through the permutation rule instead of a threshold.

Counting theorem. Under $n \geq m$, the kept ($\sigma \leq c/2$) triplets sorted descending contain the economy SVD of A in their first m slots: the augmented SVD returns M triplets, $p_\ell = \max(0, M - n)$ are pinned, and the $M - p_\ell = \min(n, M) \geq m$ survivors are the SVD of the data block — k positive values followed by exact-arithmetic zeros whose left vectors fill the data-left-null. No tolerance decides any count; m comes from the column mask, p_ℓ from the bitwise indicator.

Superseded variant. An earlier form of Method D used a second sketch Q_2 for the row space and a two-sided core $C = Q^\top A_{\text{pad}} Q_2$ with Method-A augmentation appended for both the surplus row coordinates and the pad column coordinates. It is correct, and its core-level structure (the bitwise column split of Q_2 , the exact alignment that makes core augmentation a clean scalpel) remains instructive — but the static-shape constraint exposed it as unnecessary: rebuilding V from U via Step 4 removes the second sketch, the core rotation, and the completion QR entirely. The static requirement *simplified* the algorithm.

No rank tolerance appears anywhere; the only mask-dependent runtime quantities are permutations and the bitwise indicator t . Cost: $O(NM^2 + M^3)$, independent of the pad count.

The unified permutation rule. *Before any Householder factorization, permute pad coordinates to the trailing pivot positions.* The reflector at step i is $v = x \mp \|x\| e_i$: its entries at zero rows of x vanish *except at the pivot position i* , which receives $\pm \|x\|$. An in-place pad row can therefore become a pivot; then either the trailing block is exactly zero and $H = I$ emits $q = e_{\text{pad}}$ — a null column sitting 100% on a pad, deterministically — or the trailing block is ε -level roundoff and the normalized reflector places an $O(1)$ -direction entry at the pad coordinate. Permutation ensures all pivots are data coordinates, making every exactness claim above reduce to “ $0 - 0 \cdot x = 0$ ”. This is the load-bearing step, not decoration, and it applies identically on both sides.

Worked example: the core is blind to the contamination

Take $N = 5$, $M = 3$, pad rows $\{3, 5\}$ (interior), pad column $\{3\}$, rank-1 data including a *numerically* zero data row (row 4):

$$A_{\text{pad}} = \begin{pmatrix} 3 & 6 & 0 \\ 4 & 8 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix} \begin{array}{l} \text{data} \\ \text{data} \\ \text{pad} \\ \text{data (zero)} \\ \text{pad} \end{array}$$

(entries chosen so that the QR is exact in floating point: $\|(3, 4)\| = 5$). Take $\Omega_1 = I$ for exposition. *With* permutation (row order 1, 2, 4, 3, 5): step 1 gives $q_1 = (0.6, 0.8, 0, 0, 0)^\top$; the trailing block is then exactly zero (rank 1, and column 2 is exactly twice column 1), so $H = I$ emits $q_2 = H_1 e_2 = (0.8, -0.6, 0, 0, 0)^\top$ and $q_3 = H_1 e_3 = e_4$ — the numerically-zero *data* row, which is permitted (the requirement is one-sided: avoid pads, not all zero rows). All three columns bitwise zero on rows 3 and 5. *Without* permutation: q_1, q_2 identical, but the third pivot is position 3 — a pad row — and $H = I$ emits $q_3 = e_3$ exactly: total contamination, deterministically.

Now project: in *both* cases, bit for bit,

$$B = Q^\top A_{\text{pad}} = \begin{pmatrix} 5 & 10 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix},$$

since $q_2^\top A_{\text{pad}} = 0.8 r_1 - 0.6 r_2 = 0$ exactly, and $q_3^\top A_{\text{pad}}$ is a zero row of A_{pad} under either labeling. This is the key picture: the outer pad structure reaches the core through two different channels. The pad *column* passes through faithfully (column 3 of B is bitwise zero because column 3 of A_{pad} is), which is why right-side core work is safe. But a zero *row* of B says only “this column of Q is orthogonal to $\text{col}(A_{\text{pad}})$ ” — it is blind to whether that column lives on data rows ($q_3 = e_4$, fine) or on a pad ($q_3 = e_3$, poisoned). The contamination lives entirely in Q , upstream of anything the core SVD sees. Consequently, core-level repair of a bad Q is impossible (the core contains no signal to act on, and $U = Q\bar{U}$ can only rotate pad mass among columns, not remove it): prevention by permutation at $O(NM)$ beats any repair.

Robustness

$M > \text{rank}(A)$ is the design case, not an adversarial one: capture is exact for any rank (square invertible Ω), the trailing QR columns are clean for any k (either exact $H=I$ coordinate vectors,

or reflections of data-supported roundoff), and nothing branches on k . $A_{\text{pad}} = 0$ yields coordinate-vector Q and $B = 0$, consistently (guard $c > 0$). The Haar choice of Ω removes the mildly heavy-tailed $\sigma_{\min}$ of a square Gaussian ($\kappa(\Omega) = 1$). The only genuine failure boundary is $n < m$: fewer data rows than data columns is where the *unpadded* economy SVD is itself infeasible, and it is pleasingly also exactly where the counting theorem’s inequality $p_\ell \leq p_c$ would fail. (The earlier claim that $n < M$ was the boundary was an artifact of the stronger all-columns-pad-free contract; see the output contract above.)

Companion code. Two implementations accompany this note. `pad_safe_svd.py`: a NumPy reference with an adversarial demo (interior pads inside the first M rows, rank-deficient data, a numerically-zero data row, and an $n < M$ case) and hard structural assertions — bitwise-zero pad rows of $U[:, : m]$, exact coordinate pad slots of V , unpad-equivalence against `numpy.linalg.svd` of the unpadded A . `pad_safe_svd_jax.py`: the static-shape JAX form of the final pipeline above, verified inside a `jit`-compiled `lax.scan` over a batch with a *different* mask pattern per element; the scan traces its body exactly once, and the jit cache size stays at 1 across calls with new patterns — the structural proof that mask patterns are runtime data, not compile-time shape.

9 Choosing between the methods

	A (augment)	B (completion)	C (noise)	C' (C + refine)	D (sketch-project)
Cost	$O(N(M+P_r)^2)$	$O(NM^2)$	$O(NM^2)$	$O(NM^2)$	$O(NM^2)$
Depends on pad count	yes	no	no	no	no
Exact (up to ε)	yes	yes	no	no	yes
Error level	$\sim \varepsilon$	$\sim \varepsilon$	$\sim \sqrt{\varepsilon}$	$\sim \varepsilon^{2/3}$	$\sim \varepsilon$
Rank tolerance	no	yes (τ)	no (t implicit)	no (t implicit)	no
Full SVD of big matrix	yes	yes	yes	yes	no (core only)
Post-processing	filter	QR completion	none	small SVD	QR of $A^\top U$
Pad rows of final U	$O(\varepsilon)$	exactly 0	$O(\varepsilon \ A\ / t)$	$O(\varepsilon \ A\ / t)$	exactly 0
Helps SVD derivative	no	no	yes	yes	no
Best regime	$P_r \ll n$	any P_r	fp64, loose tol	fp64/fp32, simple	default choice